Neural Networks

Most content of this lecture are based on **CS7015** course taught by Mitesh Khapra at IIT-Madras and Deep Learning for Computer Vision by Prof. Vineeth

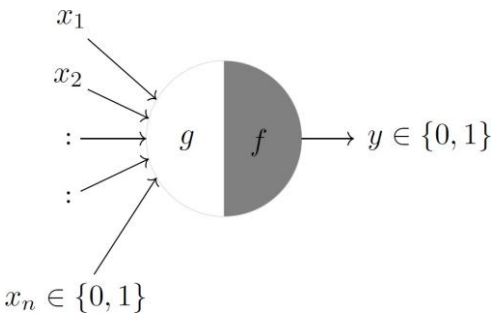
McCulloch-Pitts Neuron

- McCulloch (neuroscientist) and Pitts (logician) proposed a highly simplified computational model of the neuron (1943)
- *g* aggregates the inputs and the function *f* takes a decision based on this aggregation
- The inputs can be excitatory or inhibitory
- *y* = 0 if any x_i is inhibitory, else

$$g(x_1, x_2, \ldots, x_n) = g(\mathbf{x}) = \sum_{i=1}^{n} x_i$$

$$y = f(g(\mathbf{x})) = 1 \; if \; g(\mathbf{x}) \geq \theta$$
$$= 0 \; if \; g(\mathbf{x}) < \theta$$

θ is a thresholding parameter



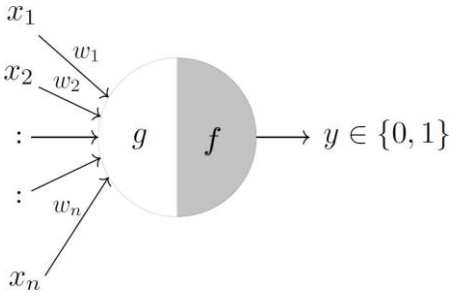
Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Perceptrons

- Frank Rosenblatt, an American psychologist, proposed the perceptron model (1958)
- Later refined and carefully analyzed by Minsky and Papert (1969)
- A more general computational model than McCulloch-Pitts neurons
- Inputs are no longer limited to boolean values

$$g(x_1, x_2, \ldots, x_n) = g(\mathbf{x}) = \sum_{i=1}^{n} w_i * x_i$$

$$y = f(g(\mathbf{x})) = 1 \; if \; g(\mathbf{x}) \geq \theta$$
$$= 0 \; if \; g(\mathbf{x}) < \theta$$



Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

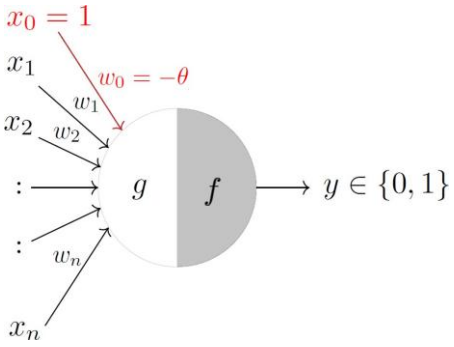
Perceptrons

- Frank Rosenblatt, an American psychologist, proposed the perceptron model (1958)
- Later refined and carefully analyzed by Minsky and Papert (1969)
- A more general computational model than McCulloch-Pitts neurons
- Inputs are no longer limited to boolean values

- A more accepted convention

$$g(x_1, x_2, \ldots, x_n) = g(\mathbf{x}) = \sum_{i=0}^{n} w_i * x_i$$

$$y = f(g(\mathbf{x})) = 1 \; if \; g(\mathbf{x}) \geq 0$$
$$= 0 \; if \; g(\mathbf{x}) < 0$$
$$where \; x_0 = 1 \; and \; w_0 = -\theta$$



Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Perceptron Learning Algorithm

Algorithm 1 Perceptron Learning

$\mathbf{w} = [w_0, w_1, w_2, \ldots, w_n]$

$\mathbf{x} = [1, x_1, x_2, \ldots, x_n]$

$P \leftarrow$ input with labels 1;

$N \leftarrow$ input with labels 0;

Initialize w randomly;

while *!convergence* **do**

  Pick random $\mathbf{x} \in P \cup N$

  if $\mathbf{x} \in P$ *and* $\sum_{i=0}^{n} w_i * x_i < 0$ **then**

    $\mathbf{w} = \mathbf{w} + \mathbf{x}$

  if $\mathbf{x} \in N$ *and* $\sum_{i=0}^{n} w_i * x_i \geq 0$ **then**

    $\mathbf{w} = \mathbf{w} - \mathbf{x}$

end

```
/* algorithm converges when all the inputs
   are classified correctly                  */
```

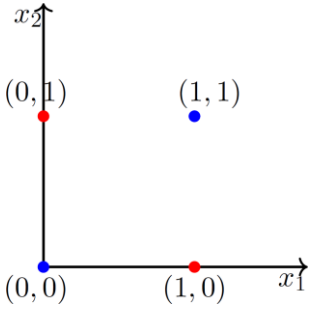
Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

The XOR Conundrum

x_1	x_2	XOR	
0	0	0	$w_0 + \sum_{i=1}^{2} w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
1	1	0	$w_0 + \sum_{i=1}^{2} w_i x_i < 0$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 < 0 \implies w_1 + w_2 < -w_0$$

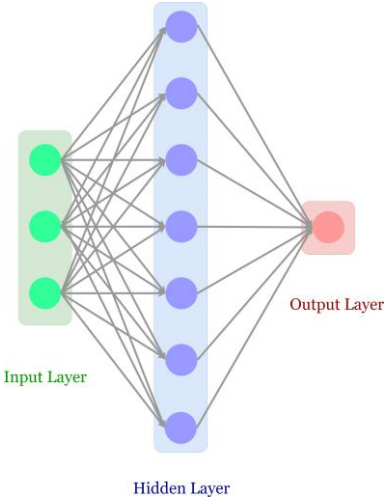
- The fourth condition contradicts conditions 2 and 3
- No solution possible satisfying this set of inequalities



- Indeed you can see that it is impossible to draw a line which separates the red points from the blue points

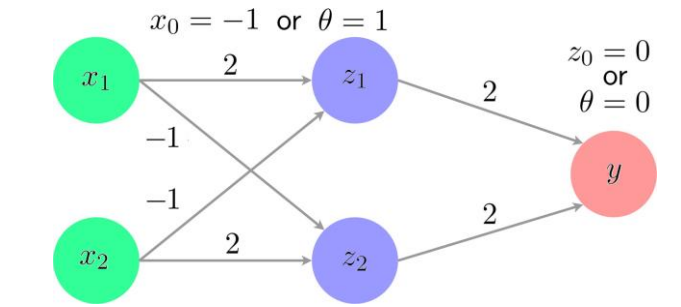
Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Multi-Layer Perceptrons



Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Solving XOR with Multi-Layer Perceptrons



(x_1,x_2)	(z_1,z_2)	y
(0,0)	(0,0)	0
(0,1)	(0,0)	1
(1,0)	(1,0)	1
(1,1)	(0,0)	0

- **Theorem:** Any boolean function of n inputs can be represented exactly by a network of perceptrons containing 1 hidden layer with 2^n perceptrons and one output layer containing 1 perceptron
- As n increases, the number of perceptrons in the hidden layers increases exponentially. We'd ideally want this to be as few as possible.

Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Going Beyond Binary Inputs and Outputs

Question

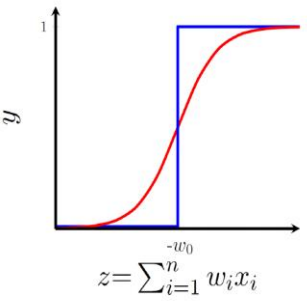
What about arbitrary functions of the form $y = f(x)$ where $x \in \mathbf{R}^n$ (instead of $\{0, 1\}^n$) and $y \in \mathbf{R}$ (instead of $\{0, 1\}$)?

Can we use the same perceptron model to represent such functions?

Need for Activation Functions

- A perceptron only fires if weighted sum of its inputs is greater than threshold $-w_0$
- Thresholding logic could be harsh at times
- E.g., when $-w_0 = 0.5$, though output values 0.49 and 0.51 are very close to each other, the perceptron would assign different labels to them.

- This behavior is not a characteristic of the problem, the weights or the threshold; it is a characteristic of the perceptron function itself which behaves like a step function
- There will always be a sudden change in decision (from 0 to 1) when $\sum_{i=1}^{n} w_i x_i$ crosses the threshold $(-w_0)$
- For most real-world applications, we'd expect a smoother decision function which gradually changes from 0 to 1

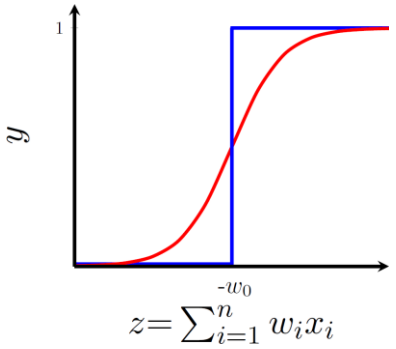


Sigmoid Neurons

- We could use any logistic function to obtain a smoother output function than a step function
- One form is the sigmoid function:

$$y = \frac{1}{1 + e^{-(w_0 + \sum_{i=0}^{n} w_i x_i)}}$$

- No longer a sharp transition at the threshold $-w_0$
- Also, output is no longer binary but a real value between 0 and 1 which can be interpreted as a probability
- Unlike the step function, this one is smooth, continuous at $-w_0$ and most importantly **differentiable**

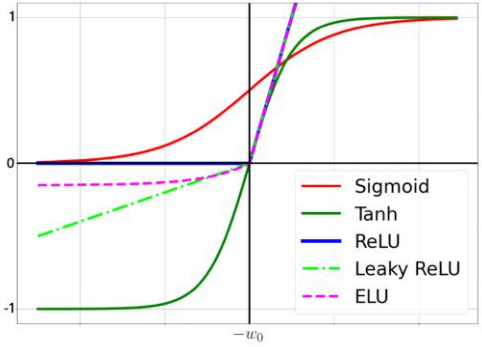


Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Other Popular Activation Functions

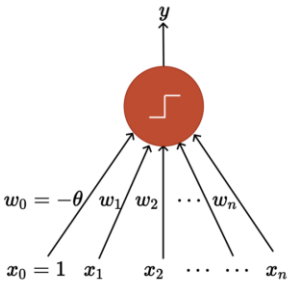
Considering $z = \sum_{i=0}^{n} w_i x_i$

- **Sigmoid:** $y = \frac{1}{1+e^{-z}}$
- **Tanh:** $y = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$
- **Rectified Linear Unit (ReLU):** $y = \max(0, z)$
- **Leaky ReLU:** $y = \max(\alpha z, z), \alpha \in (0, 1)$
- **Exponential Linear Unit (ELU):** $y = \max(\alpha(e^{z} - 1), z)$, where $\alpha > 0$



Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

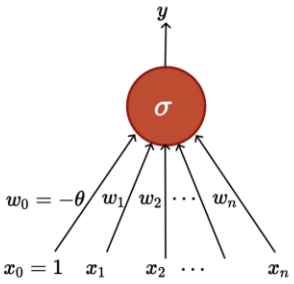
Perceptron



$$y = 1 \text{ if } \sum_{i=0}^{n} w_i x_i \geq 0$$

$$= 0 \text{ if } \sum_{i=0}^{n} w_i x_i < 0$$

Sigmoid (Logistic) Neuron



$$y = \frac{1}{1+exp\left(-(w_0+\sum_{i=1}^{n} w_i x_i)\right)}$$

Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Representation Power of MLPs

Theorem: A multilayer network of sigmoid neurons with a single hidden layer can be used to approximate any continuous function to any desired precision. (Also known as **Universal Approximation Theorem**)

Proof: Cybenko, 1989 [2] and Hornik et al, 1989[3]

[2]Cybenko, Approximation by superpositions of a sigmoidal function, Mathematics of Control, Signals and Systems, Vol 2, pp 303-314, 1989

[3]Hornik et al, Multilayer feedforward networks are universal approximators, Neural Networks Vol 2:5, pp 359-366, 1989

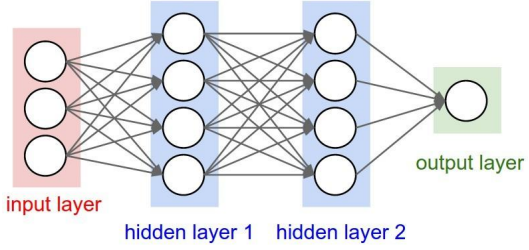
Feed Forward Neural Networks, Back Propagation

Feedforward Networks

- A feedforward neural network, also called a multi-layer perceptron, is a collection of neurons, organized in layers.

  Used to approximate some function f ∗. (f ∗ could be a classifier that maps an input vector x to a category y.

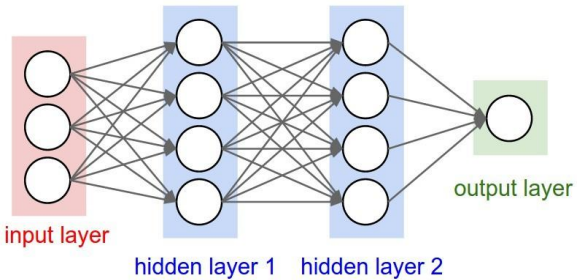
  The neurons are arranged in the form of a directed acyclic graph i.e., the information only flows in one direction - input x to output y. Hence the term feedforward.



Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Feedforward Networks

- The number of layers in the network (excluding the input layer) is known as depth
- Each neuron can be seen as a **vector**-to-**scalar** function which takes a vector of inputs from the previous layer and computes a scalar value.
- Above network can be seen as a composition of functions $y = f^{(3)}(f^{(2)}(f^{(1)}(x)))$, $f^{(1)}$:first hidden layer, $f^{(2)}$: the second , $f^{(3)}$:final output layer.



Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Feedforward Networks

- Approximate function f^*, given noisy estimates of $f^*(\boldsymbol{x})$ at different points, in the form of a dataset $\{\boldsymbol{x}_i, y_i\}_{i=1}^{M}$.
- NN defines a function $y = f(\boldsymbol{x}; \boldsymbol{\theta})$.
- The goal is to learn the parameters (weights and biases) $\boldsymbol{\theta}$ such that f best approximates f^*.

- How to find the values of the parameters i.e., train the network?

Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Training

- Neural networks are usually trained by minimizing a loss function, such as mean square error:

$$Loss_{MSE} = \frac{1}{M}\sum_{i=1}^{M}(f^*(\boldsymbol{x}) - f(\boldsymbol{x};\boldsymbol{\theta}))^2$$

- Let us consider a simple 1D example, where we try to minimize the function $f(x) = x^2$. Specifically, we find out the value x^* gives the smallest value for $f(x)$ i.e., $f(x^*)$.

$$x^* = \arg\min_x f(x)$$

Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Training

- We can obtain the slope of the function $f(x)$ at x by taking its derivative i.e., $f'(x)$.
- This means, if we give a very small *push* to x in the direction (sign) of the slope, we're sure that the function will increase.

$$f(x + p \cdot \text{sign}(f'(x))) > f(x) \quad \text{for an infinitesimally small } p$$

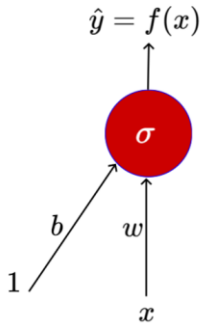
- The reverse is also true i.e.,

$$f(x - p \cdot \text{sign}(f'(x))) < f(x) \quad \text{for an infinitesimally small } p$$

- This forms the basis for gradient descent - we start off at a random x, and take small steps in the direction of the **negative** gradient.

Source: Vineet Balasubramanian, CS5370 – Deep Learning for Vision (IIT Hyderabad)

Training



$$f(x) = \frac{1}{1+e^{-(wx+b)}}$$

Input for Training

$$\{x_i, y_i\}_{i=1}^{N} \rightarrow N \text{ pairs of } (x, y)$$

Training Objective

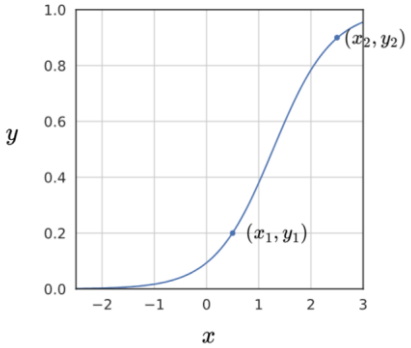
Find w and b such that

$$\underset{(w,b)\in R}{minimize}\ \mathscr{L}(w,b) = \frac{1}{N}\sum_{i=1}^{N}(y_i - f(x_i))^2$$

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Guess work

$$f(x) = \frac{1}{1+e^{-(wx+b)}}$$



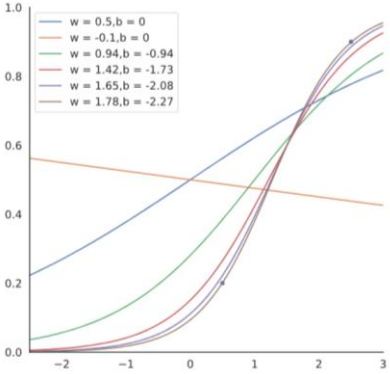
Suppose we train the network with $(x, y) = (0.5, 0.2)$ and $(2.5, 0.9)$

At the end of training we expect to find w^*, b^* such that:

$f(0.5) \rightarrow 0.2$ and $f(2.5) \rightarrow 0.9$

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

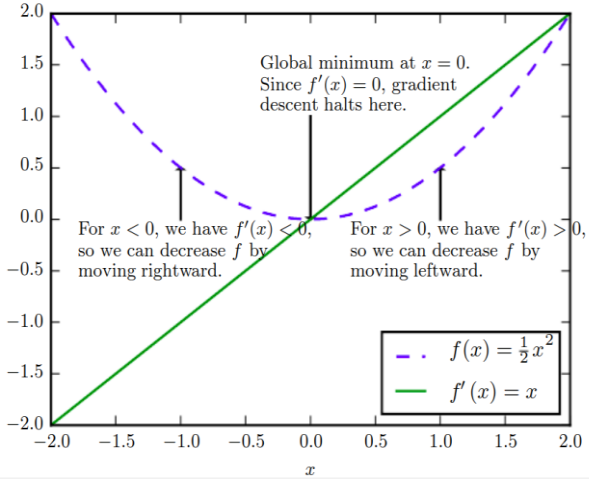
Guess work



w	b	Loss
0.50	0.00	0.0730
-0.10	0.00	0.14
0.94	-0.94	0.0214
1.42	-1.73	0.0028
1.65	-2.08	0.0003
1.78	-2.27	0.0000

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Gradient Descent: A 1D Example



Algorithm: gradient_descent()

$t \leftarrow 0$;

$max\_iterations \leftarrow 1000$;

$Initialize\ \theta_0 = [w_0, b_0]$;

while $t{+}{+} < max\_iterations$ **do**

  $\theta_{t+1} \leftarrow \theta_t - \eta \nabla \theta_t$

end

where $\nabla \theta_t = [\frac{\partial \mathcal{L}(\theta)}{\partial w_t}, \frac{\partial \mathcal{L}(\theta)}{\partial b_t}]^T$

Figure Credit: Goodfellow et al, DL Book Ch 4

Gradient Descent:

The direction u that we intend to move in should be at 180° w.r.t. the gradient

In other words, move in a direction opposite to the gradient

$$w_{t+1} = w_t - \eta \nabla w_t$$

$$b_{t+1} = b_t - \eta \nabla b_t$$

where, $\nabla w_t = \frac{\partial \mathcal{L}(w,b)}{\partial w}$, at $w = w_t, b = b_t$,

and, $\nabla b_t = \frac{\partial \mathcal{L}(w,b)}{\partial b}$, at $w = w_t, b = b_t$,

How to get ∇w, ∇b ?

Algorithm:gradient_descent()

$t \leftarrow 0$;

max_iterarions $\leftarrow 1000$;

w,b $\leftarrow$ initialize randomly

while $t <$ max_iterations **do**

  $w_{t+1} \leftarrow w_t - \eta \nabla w_t$;

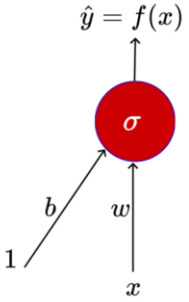
  $b_{t+1} \leftarrow b_t - \eta \nabla b_t$;

  $t \leftarrow t + 1$;

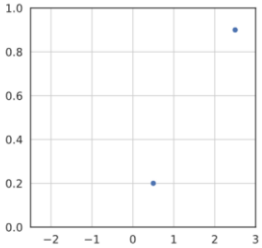
end

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Finding Gradients



$$f(x) = \frac{1}{1+e^{-(wx+b)}}$$



Let us assume there is only one point to fit

$$\mathscr{L}(w,b) = \frac{1}{2} * (f(x) - y)^2$$

$$\nabla w = \frac{\partial \mathscr{L}(w,b)}{\partial w} = \frac{\partial}{\partial w}\left[\frac{1}{2} * (f(x) - y)^2\right]$$

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Finding Gradients

$$\nabla w = \frac{\partial}{\partial w}[\frac{1}{2} * (f(x) - y)^2]$$

$$= \frac{1}{2}[2 * (f(x) - y) * \frac{\partial}{\partial w}(f(x) - y)]$$

$$= (f(x) - y) * \frac{\partial}{\partial w}(f(x))$$

$$= (f(x) - y) * \frac{\partial}{\partial w}(\frac{1}{1+e^{-(wx+b)}})$$

$$= (f(x) - y) * f(x) * (1 - f(x)) * x$$

So If we have only one point (x, y), we have,

$$\nabla w = (f(x) - y) * f(x) * (1 - f(x)) * x$$

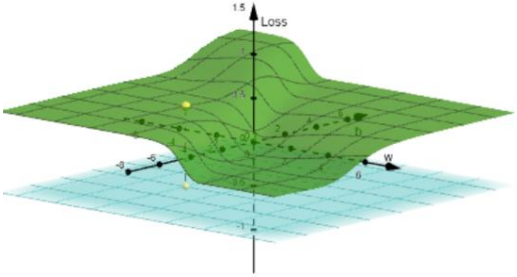
For two points,

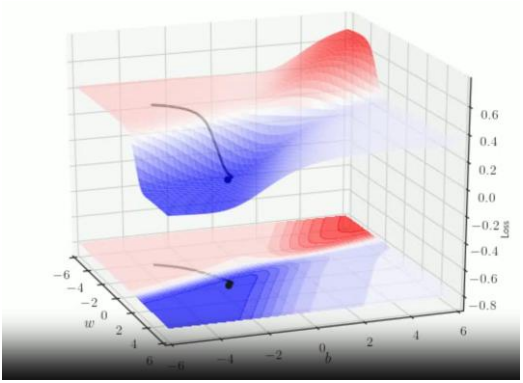
$$\nabla w = \sum_{i=1}^{2}(f(x_i) - y_i) * f(x_i) * (1 - f(x_i)) * x_i$$

$$\nabla b = \sum_{i=1}^{2}(f(x_i) - y_i) * f(x_i) * (1 - f(x_i))$$

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Finding Gradients





Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Initializing Learning rate

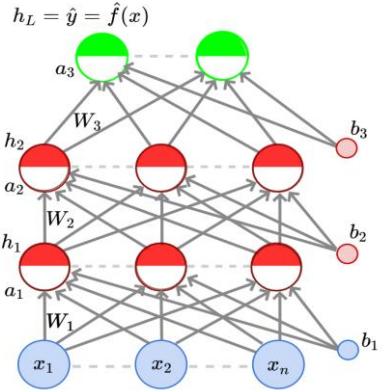
- Tune learning rate (try different values on a log scale: 0.0001, 0.001, 0.01, 0.1. 1)
- Run a few epochs with each of these and figure out a learning rate which works best
- Now do a finer search around this value (for example, if the best learning rate was 0.1 then now try some values around it: 0.05, 0.2, 0.3)

- **Exponential Decay**: with $\mathbf{n_0}$ **and k** are the hyperparameters and t is step

$$\eta = \eta_0^{-kt} \text{ where } \eta_0 \text{ and } k \text{ are}$$

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

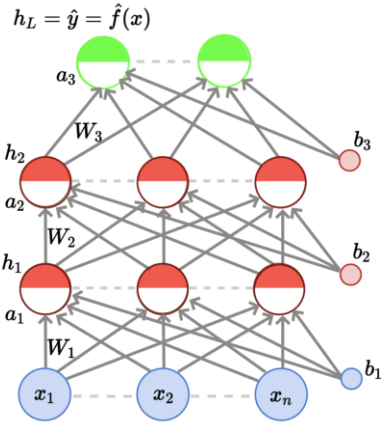
FFNN



- *L-1 hidden layers having n neurons*
- *Output layer : k neurons*
- *a_i and h_i are pre-activation and activation*

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

FFNN



The pre-activation at layer i is given by

$$a_i(x) = b_i + W_i h_{i-1}(x)$$

The activation at layer i is given by

$$h_i(x) = g(a_i(x))$$

where g is called the activation function (for example, logistic, tanh, linear, etc.)

The activation at the output layer is given by

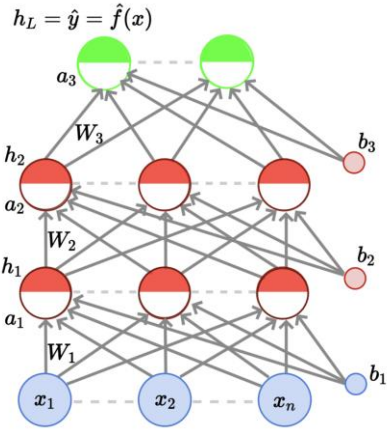
$$f(x) = h_L(x) = O(a_L(x))$$

where O is the output activation function (for example, softmax, linear, etc.)

To simplify notation we will refer to $a_i(x)$ as a_i and $h_i(x)$ as h_i

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

FFNN



Data: $\{x_i, y_i\}_{i=1}^{N}$

Model:

$$\hat{y}_i = \hat{f}(x_i) = O(W_3 g(W_2 g(W_1 x_i + b_1) + b_2) + b_3)$$

Parameters:

$$\theta = W_1, ..., W_L, b_1, b_2, ..., b_L (L = 3)$$

Algorithm: Gradient Descent with Back-propagation

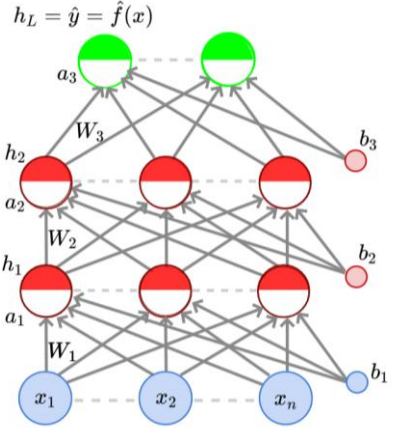
Objective/Loss/Error function: Say,

$$min \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{k} (\hat{y}_{ij} - y_{ij})^2$$

In general, $min\ \mathscr{L}(\theta)$

where $\mathscr{L}(\theta)$ is some function of the parameters

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)



Algorithm: gradient_descent()

$t \leftarrow 0;$

$max\_iterations \leftarrow 1000;$

$Initialize\ \theta_0 = [w_0, b_0];$

while $t{+}{+} < max\_iterations$ **do**

  $\theta_{t+1} \leftarrow \theta_t - \eta \nabla \theta_t$

end

where $\nabla \theta_t = [\frac{\partial \mathscr{L}(\theta)}{\partial w_t}, \frac{\partial \mathscr{L}(\theta)}{\partial b_t}]^T$

Now, in this feedforward neural network, instead of $\theta = [w, b]$ we have $\theta = [W_1, ..., W_L, b_1, b_2, ..., b_L]$

We can still use the same algorithm for learning the parameters of our model

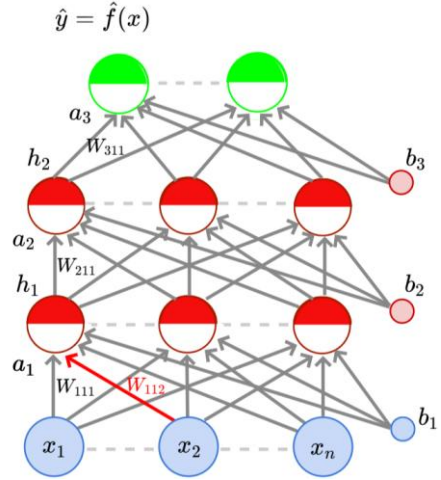
Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Backpropagation

Let us focus on this one weight (W_{112}).

To learn this weight using SGD we need a formula for $\frac{\partial \mathscr{L}(\theta)}{\partial W_{112}}$.

We will see how to calculate this.



Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

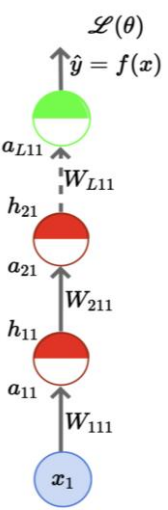
Chain rule for derivatives

$$\frac{\partial \mathscr{L}(\theta)}{\partial W_{111}} = \frac{\partial \mathscr{L}(\theta)}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{L11}} \frac{\partial a_{L11}}{\partial h_{21}} \frac{\partial h_{21}}{\partial a_{21}} \frac{\partial a_{21}}{\partial h_{11}} \frac{\partial h_{11}}{\partial a_{11}} \frac{\partial a_{11}}{\partial W_{111}}$$

$$\frac{\partial \mathscr{L}(\theta)}{\partial W_{111}} = \frac{\partial \mathscr{L}(\theta)}{\partial h_{11}} \frac{\partial h_{11}}{\partial W_{111}}$$ (just compressing the chain rule)

$$\frac{\partial \mathscr{L}(\theta)}{\partial W_{211}} = \frac{\partial \mathscr{L}(\theta)}{\partial h_{21}} \frac{\partial h_{21}}{\partial W_{211}}$$

$$\frac{\partial \mathscr{L}(\theta)}{\partial W_{L11}} = \frac{\partial \mathscr{L}(\theta)}{\partial a_{L1}} \frac{\partial a_{L1}}{\partial W_{L11}}$$

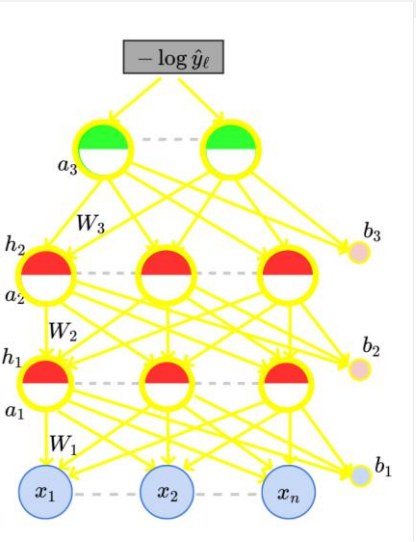


Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Chain rule for derivatives

- Gradient w.r.t. output units
- Gradient w.r.t. hidden units
- Gradient w.r.t. weights and biases

$$\underbrace{\frac{\partial \mathcal{L}(\theta)}{\partial W_{111}}}_{} = \underbrace{\frac{\partial \mathcal{L}(\theta)}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_3}}_{} \underbrace{\frac{\partial a_3}{\partial h_2} \frac{\partial h_2}{\partial a_2}}_{} \underbrace{\frac{\partial a_2}{\partial h_1} \frac{\partial h_1}{\partial a_1}}_{} \underbrace{\frac{\partial a_1}{\partial W_{111}}}_{}$$



Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Chain rule for derivatives

Lets take a simple example of a $W_k \in \mathbb{R}^{3\times 3}$ and see what each entry looks like

$$\nabla_{W_k}\mathscr{L}(\theta) = \begin{bmatrix} \frac{\partial \mathscr{L}(\theta)}{\partial W_{k11}} & \frac{\partial \mathscr{L}(\theta)}{\partial W_{k12}} & \frac{\partial \mathscr{L}(\theta)}{\partial W_{k13}} \\ \frac{\partial \mathscr{L}(\theta)}{\partial W_{k21}} & \frac{\partial \mathscr{L}(\theta)}{\partial W_{k22}} & \frac{\partial \mathscr{L}(\theta)}{\partial W_{k23}} \\ \frac{\partial \mathscr{L}(\theta)}{\partial W_{k31}} & \frac{\partial \mathscr{L}(\theta)}{\partial W_{k32}} & \frac{\partial \mathscr{L}(\theta)}{\partial W_{k33}} \end{bmatrix} \quad \frac{\partial \mathscr{L}(\theta)}{\partial W_{kij}} = \frac{\partial \mathscr{L}(\theta)}{\partial a_{ki}} \frac{\partial a_{ki}}{\partial W_{kij}}$$

$$\nabla_{W_k}\mathscr{L}(\theta) = \begin{bmatrix} \frac{\partial \mathscr{L}(\theta)}{\partial a_{k1}} h_{k-1,1} & \frac{\partial \mathscr{L}(\theta)}{\partial a_{k1}} h_{k-1,2} & \frac{\partial \mathscr{L}(\theta)}{\partial a_{k1}} h_{k-1,3} \\ \frac{\partial \mathscr{L}(\theta)}{\partial a_{k2}} h_{k-1,1} & \frac{\partial \mathscr{L}(\theta)}{\partial a_{k2}} h_{k-1,2} & \frac{\partial \mathscr{L}(\theta)}{\partial a_{k2}} h_{k-1,3} \\ \frac{\partial \mathscr{L}(\theta)}{\partial a_{k3}} h_{k-1,1} & \frac{\partial \mathscr{L}(\theta)}{\partial a_{k3}} h_{k-1,2} & \frac{\partial \mathscr{L}(\theta)}{\partial a_{k3}} h_{k-1,3} \end{bmatrix} = \nabla_{a_k}\mathscr{L}(\theta) \cdot \mathrm{h}_{\mathrm{k-1}}^{T}$$

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Chain rule for derivatives

Finally, coming to the biases

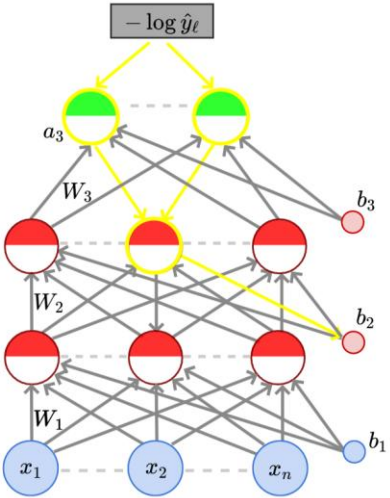
$$a_{ki} = b_{ki} + W_{kij} h_{k-1,j}$$

$$\frac{\partial \mathscr{L}(\theta)}{\partial b_{ki}} = \frac{\partial \mathscr{L}(\theta)}{\partial a_{ki}} \frac{\partial a_{ki}}{\partial b_{ki}}$$

$$= \frac{\partial \mathscr{L}(\theta)}{\partial a_{ki}}$$

We can now write the gradient w.r.t. the vector b_k

$$\nabla_{b_k} \mathscr{L}(\theta) = \begin{bmatrix} \frac{\partial \mathscr{L}(\theta)}{\partial a_{k1}} \\ \frac{\partial \mathscr{L}(\theta)}{\partial a_{k2}} \\ \vdots \\ \frac{\partial \mathscr{L}(\theta)}{\partial a_{kn}} \end{bmatrix} = \nabla_{a_k} \mathscr{L}(\theta)$$



Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Computing Gradients

$$\nabla_{a_L} \mathcal{L}(\theta) \qquad \text{(gradient w.r.t. output layer)}$$

$$\nabla_{h_k} \mathcal{L}(\theta), \nabla_{a_k} \mathcal{L}(\theta) \qquad \text{(gradient w.r.t. hidden layers, } 1 \leq k < L\text{)}$$

$$\nabla_{W_k} \mathcal{L}(\theta), \nabla_{b_k} \mathcal{L}(\theta) \qquad \text{(gradient w.r.t. weights and biases, } 1 \leq k < L\text{)}$$

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Algorithm for Gradient Descent

Algorithm: gradient_descent()

$t \leftarrow 0;$

$max\_iterations \leftarrow 1000;$

$Initialize\ \theta_0 = [W_1^0, ..., W_L^0, b_1^0, ..., b_L^0];$

while $t{+}{+} < max\_iterations$ **do**

$a_1, h_1, a_2, h_2, ..., a_{L-1}, h_{L-1}, a_L, \hat{y} = forward\_propagation(\theta_t)$

$\nabla\theta_t = backward\_propagation(h_1, h_2, ..., h_{L-1}, a_1, a_2, ..., a_L, y, \hat{y})$

$\theta_{t+1} \leftarrow \theta_t - \eta\nabla\theta_t$

end

Algorithm: forward_propagation(θ)

for $k = 1$ to $L - 1$ **do**

$a_k = b_k + W_k h_{k-1};$

$h_k = g(a_k)$

end

$a_L = b_L + W_L h_{L-1};$

$\hat{y} = O(a_L);$

Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Algorithm for Gradient Descent

Just do a forward propagation and compute all h_i's, a_i's, and $\hat{y}$

Algorithm: back_propagation($h_1, h_2, ..., h_{L-1}, a_1, a_2, ..., a_L, y, \hat{y}$)

// Compute output gradient ;

$\nabla_{a_L} \mathscr{L}(\theta) = -(e(y) - \hat{y})$;

for $k = L$ to 1 **do**

  // Compute gradients w.r.t. parameters ;

  $\nabla_{W_k} \mathscr{L}(\theta) = \nabla_{a_k} \mathscr{L}(\theta) h_{k-1}^T$;

  $\nabla_{b_k} \mathscr{L}(\theta) = \nabla_{a_k} \mathscr{L}(\theta)$;

  // Compute gradients w.r.t. layer below ;

  $\nabla_{h_{k-1}} \mathscr{L}(\theta) = W_k^T \nabla_{a_k} \mathscr{L}(\theta)$;

  // Compute gradients w.r.t. layer below (pre-activation) ;

  $\nabla_{a_{k-1}} \mathscr{L}(\theta) = \nabla_{h_{k-1}} \mathscr{L}(\theta) \odot [..., g'(a_{k-1,j}), ...]$;

end

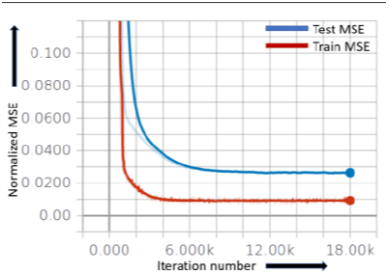
Source: Mitesh Khapra, CS7015 – Deep Learning (IIT Madras)

Train and Test Errors

- We need to compute error after each epoch or batch and find out the error, then back propagate
- Train error trend tells us how the model is learning
- Test error model performance on unseen data

$$train_{err} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{f}(x_i))^2$$

$$test_{err} = \frac{1}{m} \sum_{i=n+1}^{n+m} (y_i - \hat{f}(x_i))^2$$



References

- Please refer to Mitesh Khapra's original lecture slides (and videos) for more detailed explanation of some of these topics, available at CS7015: Deep Learning.
- Also refer to Vineet Balasubramanian original lecture slide and videos for more detailed information CS5370: Deep Learning for Vision

Other good resources:

Deep Learning Book: Chapter 6 for Multilayer Perceptrons
Stanford CS231n Notes
Stanford UFLDL tutorial on Multilayer Neural Networks
Neural Networks: A Systematic Introduction by Raúl Rojas